

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN 2

Đề tài: Data Fitting và Phương Pháp OLS

Môn học: Toán Ứng dụng và Thống kê

Sinh viên thực hiện:

23120017 - Trần Thanh An

23120096 - Nguyễn Viết Toàn

23120303 - Nguyễn Hồng Ngọc

24120278 - Võ Văn Khánh Đăng

24120472 - Trương Huệ Trí

Giáo viên hướng dẫn:

Võ Nam Thực Doan

Lê Nhật Nam

Ngày 9 tháng 6 năm 2026



Mục lục

Bảng đánh giá mức độ hoàn thành và phân công công việc	1
1 Phần 1: Lý Thuyết Data Fitting và Minh Họa	2
1.1 Bài Toán Data Fitting	2
1.1.1 Phát biểu bài toán tổng quát	2
1.1.2 Các Giả Thiết Gauss–Markov	2
1.2 Phương Pháp Ordinary Least Squares (OLS)	2
1.2.1 Hàm mất mát và nghiệm OLS	2
1.2.2 Ma Trận Chiều và Hat Matrix	3
1.2.3 Định Lý Gauss–Markov	4
1.2.4 Ước Lượng Phương Sai Nhiễu	4
1.3 Đánh Giá Mô Hình	4
1.3.1 Phân rã tổng bình phương	4
1.3.2 Hệ số xác định R^2 và $\bar{R}^2$	4
1.3.3 Kiểm Định Giả Thuyết	5
1.4 Các Vấn Đề Nâng Cao trong Data Fitting	5
1.4.1 Đa cộng tuyến (Multicollinearity)	5
1.4.2 Hồi Quy Ridge và Lasso (Regularization)	5
1.4.3 Phân Tích Phần Dư (Residual Analysis)	6
1.4.4 Cross-Validation và Lựa Chọn Mô Hình	6
1.5 Hàm <code>ols_fit(X, y)</code>	6
1.5.1 Cơ sở Lý thuyết & Công thức Toán học	6
1.5.2 Kết quả Cài đặt & Kiểm chứng	6
1.6 Hàm <code>hat_matrix(X)</code>	17
1.6.1 Cơ sở Lý thuyết & Công thức Toán học	17
1.7 Hàm <code>model_metrics(y, y_hat, p)</code>	17
1.7.1 Cơ sở Lý thuyết & Công thức Toán học	17
1.8 Hàm <code>coef_inference(X, y, alpha, 2)</code>	17
1.8.1 Cơ sở Lý thuyết & Công thức Toán học	17
1.9 Hàm <code>vif(X)</code> – Hệ số Phóng đại Phương sai	18

1.9.1	Cơ sở Lý thuyết & Công thức Toán học	18
1.9.2	Kết quả Cài đặt & Kiểm chứng	18
1.10	Hồi quy Ridge – Hàm <code>ridge_fit(X, y, lam)</code>	19
1.10.1	Cơ sở Lý thuyết & Công thức Toán học	19
1.10.2	Kết quả Cài đặt & Kiểm chứng	19
1.11	Biểu đồ Phân tích Phần dư – <code>residual_plots</code>	22
1.11.1	Cơ sở Lý thuyết & Công thức Toán học	22
1.11.2	Kết quả Cài đặt & Kiểm chứng	22
1.12	Kiểm định chéo K-Fold – <code>kfold_cv(X, y, k)</code>	24
1.12.1	Cơ sở Lý thuyết & Công thức Toán học	24
1.12.2	Kết quả Cài đặt & Kiểm chứng	24
1.13	Minh họa Định lý Gauss–Markov (Monte Carlo)	25
1.13.1	Cơ sở Lý thuyết & Công thức Toán học	25
1.13.2	Kết quả Cài đặt Mô phỏng (Monte Carlo)	25
2	Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế	28
2.1	Giới Thiệu Bài Toán	28
2.1.1	Lý do chọn dataset	28
2.1.2	Mô tả features	29
2.2	Phân Tích Dữ Liệu (EDA)	29
2.2.1	Thống kê mô tả	29
2.2.2	Phân tích missing values	30
2.2.3	Phân tích cơ chế missing và lý do xử lý	30
2.2.4	Phát hiện outlier	31
2.2.5	Phân tích tương quan	32
2.3	Tiền Xử Lý Dữ Liệu	33
2.3.1	Pipeline tổng thể	33
2.3.2	Winsorization và Log1p Transform	34
2.3.3	Feature Encoding	35
2.3.4	Class DataPipeline — Tránh Data Leakage	35
2.4	Xây Dựng Mô Hình	36

2.4.1	M1 — OLS Full (Baseline)	36
2.4.2	M2 — OLS Selected (p-value + VIF)	36
2.4.3	M3 — Ridge (L2) và Lasso (L1)	36
2.4.4	M4 — Polynomial Features + Ridge (Bonus)	37
2.4.5	M5b — Bayesian Ridge (Bonus)	37
2.5	Đánh Giá và So Sánh Mô Hình	38
2.5.1	Bảng tổng hợp	38
2.5.2	Phân tích phần dư (Ridge — mô hình tuyến tính tốt nhất)	39
2.5.3	Feature Importance	41
2.6	Unit Tests	42
2.7	Kết Luận	42
2.7.1	Tóm tắt kết quả	42
2.7.2	Bài học rút ra	42
2.7.3	Hướng mở rộng	43
2.7.4	Hạn chế	43
3	Tài Liệu Tham Khảo	44

Bảng đánh giá mức độ hoàn thành và phân công công việc

MSSV	Họ và Tên	Công việc thực hiện	Hoàn thành	% Đóng góp
23120017	Trần Thanh An	Cài đặt phần 1 (vif, ridge_fit, residual_plots, kfold_cv, Minh họa định lý Gauss–Markov)	100%	25%
23120096	Nguyễn Viết Toàn	Trưởng nhóm, Tìm dataset, tổng hợp source code và làm báo cáo	100%	25%
23120303	Nguyễn Hồng Ngọc	Không tham gia làm	0%	0%
24120278	Võ Văn Khánh Đăng	Cài đặt Phần 1 (ols_fit, hat_matrix, model_metrics, coef_inference)	100%	25%
24120472	Trương Huệ Trí	Cài đặt Phần 2	100%	25%

1 Phần 1: Lý Thuyết Data Fitting và Minh Họa

1.1 Bài Toán Data Fitting

1.1.1 Phát biểu bài toán tổng quát

Cho tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $\mathbf{x}_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$. Mô hình hồi quy tuyến tính giả thiết quan hệ:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \varepsilon_i, \quad (1)$$

trong đó $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^\top \in \mathbb{R}^{p+1}$ là vector tham số cần ước lượng và ε_i là nhiễu ngẫu nhiên. Dưới dạng ma trận, với ma trận design $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ (cột đầu toàn số 1):

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}. \quad (2)$$

1.1.2 Các Giả Thiết Gauss–Markov

- GM1 – Tuyến tính: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$.
- GM2 – Không hoàn hảo đa cộng tuyến: $\text{rank}(\mathbf{X}) = p + 1$.
- GM3 – Ngoại sinh: $\mathbb{E}[\boldsymbol{\varepsilon} \mid \mathbf{X}] = \mathbf{0}$.
- GM4 – Đồng phương sai: $\text{Var}(\boldsymbol{\varepsilon} \mid \mathbf{X}) = \sigma^2 \mathbf{I}_n$.
- GM5 – Phần dư chuẩn: $\boldsymbol{\varepsilon} \mid \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$.

1.2 Phương Pháp Ordinary Least Squares (OLS)

1.2.1 Hàm mất mát và nghiệm OLS

Phương pháp OLS tìm $\hat{\boldsymbol{\beta}}$ tối thiểu hóa tổng bình phương phần dư (Residual Sum of Squares – RSS):

$$\text{RSS}(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 = \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2. \quad (3)$$

[Nghiệm OLS – Normal Equations] Nếu $\mathbf{X}^\top \mathbf{X}$ khả nghịch, nghiệm OLS duy nhất là:

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (4)$$

Tính gradient của hàm mất mát theo β :

$$\nabla_{\beta} \text{RSS} = -2\mathbf{X}^{\top}(\mathbf{y} - \mathbf{X}\beta).$$

Đặt gradient bằng không:

$$\mathbf{X}^{\top}\mathbf{X}\hat{\beta} = \mathbf{X}^{\top}\mathbf{y}.$$

Vì $\mathbf{X}^{\top}\mathbf{X}$ khả nghịch, nhân trái bởi $(\mathbf{X}^{\top}\mathbf{X})^{-1}$:

$$\hat{\beta}_{\text{OLS}} = (\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top}\mathbf{y}.$$

Hessian $\nabla_{\beta}^2 \text{RSS} = 2\mathbf{X}^{\top}\mathbf{X}$ là ma trận xác định dương (khi $\text{rank}(\mathbf{X}) = p + 1$), nên nghiệm trên là cực tiểu toàn cục duy nhất.

1.2.2 Ma Trận Chiều và Hat Matrix

[Hat Matrix] Ma trận chiều (hat matrix) được định nghĩa là:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top} \in \mathbb{R}^{n \times n}. \quad (5)$$

Tên gọi xuất phát từ việc $\mathbf{H}$ “đội mũ” lên $\mathbf{y}$: $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$.

Các tính chất của $\mathbf{H}$:

1. *Idempotent*: $\mathbf{H}^2 = \mathbf{H}$.
2. *Đối xứng*: $\mathbf{H}^{\top} = \mathbf{H}$.
3. Giá trị riêng chỉ là 0 hoặc 1.
4. $\text{rank}(\mathbf{H}) = p + 1$.
5. Phần dư: $\hat{\epsilon} = (\mathbf{I} - \mathbf{H})\mathbf{y}$.

[Chứng minh $\mathbf{H}^2 = \mathbf{H}$]

$$\mathbf{H}^2 = [\mathbf{X}(\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top}][\mathbf{X}(\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top}] = \mathbf{X}(\mathbf{X}^{\top}\mathbf{X})^{-1} \underbrace{\mathbf{X}^{\top}\mathbf{X}^{-1}}_{=\mathbf{I}} \mathbf{X}^{\top} = \mathbf{H}.$$

1.2.3 Định Lý Gauss–Markov

[Gauss–Markov – BLUE] Dưới các giả thiết GM1–GM4:

1. **Không chệch:** $\mathbb{E}[\hat{\beta}_{OLS} | \mathbf{X}] = \beta$.
2. **Ma trận hiệp phương sai:** $\text{Var}(\hat{\beta}_{OLS} | \mathbf{X}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$.
3. **BLUE:** Trong lớp các ước lượng tuyến tính không chệch, OLS có phương sai nhỏ nhất.

1.2.4 Ước Lượng Phương Sai Nhiều

Sau khi có $\hat{\beta}_{OLS}$, ước lượng không chệch của σ^2 là:

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - p - 1} = \frac{\|\mathbf{y} - \mathbf{X}\hat{\beta}\|^2}{n - p - 1}. \quad (6)$$

Ta chia cho $(n - p - 1)$ vì đã ước lượng $(p + 1)$ tham số, làm mất đi $(p + 1)$ bậc tự do. Có thể chứng minh $\mathbb{E}[\hat{\sigma}^2] = \sigma^2$.

1.3 Đánh Giá Mô Hình

1.3.1 Phân rã tổng bình phương

Gọi $\bar{y} = \frac{1}{n} \sum y_i$. Ta có phân rã:

$$\underbrace{\sum (y_i - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum (\hat{y}_i - \bar{y})^2}_{\text{ESS}} + \underbrace{\sum (y_i - \hat{y}_i)^2}_{\text{RSS}}. \quad (7)$$

1.3.2 Hệ số xác định R^2 và $\bar{R}^2$

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{ESS}}{\text{TSS}} \in [0, 1]. \quad (8)$$

R^2 không bao giờ giảm khi thêm biến. Để so sánh mô hình với số biến khác nhau, ta dùng R^2 hiệu chỉnh:

$$\bar{R}^2 = 1 - \frac{n - 1}{n - p - 1}(1 - R^2). \quad (9)$$

1.3.3 Kiểm Định Giả Thuyết

Kiểm định t (Student's t -test). Giả thuyết $H_0 : \beta_j = 0$ (biến X_j không có ý nghĩa trong mô hình). Dưới GM5, $\hat{\beta}_{OLS} | \mathbf{X} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1})$. Thống kê kiểm định:

$$t_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)} \sim t_{n-p-1} \quad (\text{dưới } H_0), \quad (10)$$

với $SE(\hat{\beta}_j) = \hat{\sigma} \sqrt{[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj}}$. Khoảng tin cậy 95%:

$$CI_{95}(\beta_j) = [\hat{\beta}_j - t_{\alpha/2, n-p-1} \cdot SE(\hat{\beta}_j), \hat{\beta}_j + t_{\alpha/2, n-p-1} \cdot SE(\hat{\beta}_j)]. \quad (11)$$

Kiểm định F cho mô hình tổng thể. $H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$.

$$F = \frac{(TSS - RSS)/p}{RSS/(n - p - 1)} \sim F_{p, n-p-1}. \quad (12)$$

1.4 Các Vấn Đề Nâng Cao trong Data Fitting

1.4.1 Đa cộng tuyến (Multicollinearity)

Đa cộng tuyến xảy ra khi các cột của $\mathbf{X}$ có tương quan cao, khiến $\mathbf{X}^\top \mathbf{X}$ gần suy biến. Hệ số phóng đại phương sai (VIF):

$$VIF_j = \frac{1}{1 - R_j^2}, \quad (13)$$

trong đó R_j^2 là R^2 khi hồi quy biến X_j theo các biến còn lại. $VIF > 10$ cho thấy đa cộng tuyến nghiêm trọng.

1.4.2 Hồi Quy Ridge và Lasso (Regularization)

Ridge Regression (L2):

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|_2^2 \} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (14)$$

Lasso Regression (L1):

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|_1 \}. \quad (15)$$

Lasso không có nghiệm closed-form; giải bằng coordinate descent hoặc subgradient methods.

1.4.3 Phân Tích Phần Dư (Residual Analysis)

Bốn biểu đồ chẩn đoán chuẩn mực:

1. **Residuals vs Fitted:** Kiểm tra tính tuyến tính và đồng phương sai.
2. **Normal Q-Q:** Kiểm tra tính chuẩn của phần dư.
3. **Scale-Location:** Kiểm tra homoscedasticity.
4. **Cook's Distance:** Xác định các quan sát ảnh hưởng lớn. Công thức: $D_i = \frac{r_i^2}{p} \cdot \frac{h_{ii}}{1 - h_{ii}}$, với h_{ii} là đường chéo của $\mathbf{H}$, r_i là phần dư chuẩn hóa.

1.4.4 Cross-Validation và Lựa Chọn Mô Hình

k -Fold Cross-Validation:

$$CV(k) = \frac{1}{k} \sum_{i=1}^k MSE_i. \quad (16)$$

Tiêu chí lựa chọn mô hình:

$$AIC = n \ln \left(\frac{RSS}{n} \right) + 2(p + 2), \quad BIC = n \ln \left(\frac{RSS}{n} \right) + (p + 2) \ln n. \quad (17)$$

1.5 Hàm `ols_fit(X, y)`

1.5.1 Cơ sở Lý thuyết & Công thức Toán học

Hàm `ols_fit` ước lượng hệ số hồi quy OLS theo công thức $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ và tính phương sai nhiễu $\hat{\sigma}^2 = RSS/(n - p - 1)$. Các hàm ma trận hỗ trợ được cài đặt từ đầu gồm: `mat_zeros`, `mat_transpose`, `mat_mul`, `mat_vec_mul`, `mat_inverse`, `mat_identity`, `add_intercept`.

1.5.2 Kết quả Cài đặt & Kiểm chứng

Hàm trả về dictionary gồm: `beta_hat`, `sigma2`, `rss`, `y_hat`, `residuals`, `n`, `p`, `dof`. Năm unit test được chạy và so sánh với `numpy.linalg.lstsq`. Ở đây kiểm tra cả 3 hàm `ols_fit`, `hat_matrix`, `model_metrics` tại đây, do 3 hàm đều có input giống nhau, nên thực hiện 5 unit test chung input.

- **Unit Test 1:** $y = -1.5 + 1.5x$, dữ liệu 6 điểm.
- **Unit Test 2:** $y = 2 + 3x$, dữ liệu 5 điểm (fit hoàn hảo).
- **Unit Test 3:** $y = 2x$ (không intercept), 4 điểm.
- **Unit Test 4:** $y = 1 + 2x_1 + 0.5x_2$, hồi quy bội 6 điểm.
- **Unit Test 5:** $y \approx 2x$, dữ liệu có nhiễu, 6 điểm.

```

Test 1: y = -1.5 + 1.5*x
-----Function 1-----
  beta_hat:
    intercept = -1.500000
           x = +1.500000
  sigma^2      = 0.625000
-----Function 2-----
  Hat Matrix H =
    [ +0.4643 +0.3452 +0.2857 +0.1071 -0.0119 -0.1905 ]
    [ +0.3452 +0.2738 +0.2381 +0.1310 +0.0595 -0.0476 ]
    [ +0.2857 +0.2381 +0.2143 +0.1429 +0.0952 +0.0238 ]
    [ +0.1071 +0.1310 +0.1429 +0.1786 +0.2024 +0.2381 ]
    [ -0.0119 +0.0595 +0.0952 +0.2024 +0.2738 +0.3810 ]
    [ -0.1905 -0.0476 +0.0238 +0.2381 +0.3810 +0.5952 ]

Check H^2 = H : H satisfies the idempotent property
-----Function 3-----
  RSS      : 2.500000
  TSS      : 191.500000
  R^2      : 0.986945
  Adjusted R^2 : 0.983681
  F-statistic : 302.400000
-----Function 4-----

      beta      SE      t-stat      p-value
-----
    -1.5000    0.6099    -2.4593    0.0697
     1.5000    0.0863    17.3897    0.0001
  
```

Hình 1: Kết quả Unit Test 1: $y = -1.5 + 1.5x$. Hàm `ols_fit` cho $\hat{\beta}_0 = -1.5$, $\hat{\beta}_1 = 1.5$, $\hat{\sigma}^2 = 0.625$.

```
--- Kiểm chứng NumPy: Test 1:  $y = -1.5 + 1.5 \cdot x$  ---  
[PASS] ols_fit beta_hat  
[PASS] ols_fit y_hat  
[PASS] ols_fit rss  
[PASS] ols_fit sigma2  
[PASS] hat_matrix H  
[PASS] hat_matrix H^2=H  
[PASS] model_metrics rss  
[PASS] model_metrics r2  
[PASS] model_metrics r2_adj  
[PASS] model_metrics f_stat  
[PASS] coef_inference se  
[PASS] coef_inference p_values  
[PASS] coef_inference t_stats  
[PASS] coef_inference ci_lower  
[PASS] coef_inference ci_upper
```

Hình 2: Kiểm chứng NumPy – Test 1: tất cả các mục đều [PASS].

```

Test 2: y = 2 + 3*x
-----Function 1-----
  beta_hat:
    intercept = +2.000000
    x = +3.000000
  sigma^2      = 0.000000
-----Function 2-----
  Hat Matrix H =
    [ +0.6000 +0.4000 +0.2000 +0.0000 -0.2000 ]
    [ +0.4000 +0.3000 +0.2000 +0.1000 -0.0000 ]
    [ +0.2000 +0.2000 +0.2000 +0.2000 +0.2000 ]
    [ -0.0000 +0.1000 +0.2000 +0.3000 +0.4000 ]
    [ -0.2000 +0.0000 +0.2000 +0.4000 +0.6000 ]

Check H^2 = H : H satisfies the idempotent property
-----Function 3-----
  RSS      : 0.000000
  TSS      : 90.000000
  R^2      : 1.000000
  Adjusted R^2 : 1.000000
  F-statistic : inf
-----Function 4-----

      beta      SE      t-stat      p-value
-----
      2.0000      0.0000      inf      0.0000
      3.0000      0.0000      inf      0.0000

```

Hình 3: Kết quả Unit Test 2: $y = 2 + 3x$ (fit hoàn hảo, $\hat{\sigma}^2 = 0$, $R^2 = 1$, $F\text{-stat} = \infty$).

```
--- Kiểm chung NumPy: Test 2:  $y = 2 + 3x$  ---  
[SKIP] coef_inference t/CI ( $\sigma^2 \sim 0$ , fit hoàn hảo)  
[PASS] ols_fit beta_hat  
[PASS] ols_fit y_hat  
[PASS] ols_fit rss  
[PASS] ols_fit sigma2  
[PASS] hat_matrix H  
[PASS] hat_matrix  $H^2=H$   
[PASS] model_metrics rss  
[PASS] model_metrics r2  
[PASS] model_metrics r2_adj  
[PASS] model_metrics f_stat  
[PASS] coef_inference se  
[PASS] coef_inference p_values
```

Hình 4: Kiểm chứng NumPy – Test 2: tất cả các mục [PASS].

```

Test 3: y = 2*x (no intercept)
-----Function 1-----
  beta_hat:
    intercept = +2.000000
    sigma^2    = 0.000000
-----Function 2-----
  Hat Matrix H =
    [ +0.0333 +0.0667 +0.1000 +0.1333 ]
    [ +0.0667 +0.1333 +0.2000 +0.2667 ]
    [ +0.1000 +0.2000 +0.3000 +0.4000 ]
    [ +0.1333 +0.2667 +0.4000 +0.5333 ]

Check H^2 = H : H satisfies the idempotent property
-----Function 3-----
  RSS          : 0.000000
  TSS          : 20.000000
  R^2          : 1.000000
  Adjusted R^2 : 1.000000
  F-statistic  : inf
-----Function 4-----

      beta      SE      t-stat      p-value
-----
      2.0000      0.0000      inf      0.0000
  
```

Hình 5: Kết quả Unit Test 3: $y = 2x$ (không intercept, fit hoàn hảo).

```
--- Kiểm chứng NumPy: Test 3:  $y = 2 \cdot x$  (no intercept) ---  
D:\Data-Fitting-and-OLS\part1\unit_test.py:89: RuntimeWarning:  
  t_stats = beta / se  
[SKIP] coef_inference t/CI ( $\sigma^2 \sim 0$ , fit hoàn hảo)  
[PASS] ols_fit beta_hat  
[PASS] ols_fit y_hat  
[PASS] ols_fit rss  
[PASS] ols_fit sigma2  
[PASS] hat_matrix H  
[PASS] hat_matrix  $H^2=H$   
[PASS] model_metrics rss  
[PASS] model_metrics r2  
[PASS] model_metrics r2_adj  
[PASS] model_metrics f_stat  
[PASS] coef_inference se  
[PASS] coef_inference p_values
```

Hình 6: Kiểm chứng NumPy – Test 3.

```

Test 4:  $y = 1 + 2x_1 + 0.5x_2$ 
-----Function 1-----
  beta_hat:
    intercept = +1.000000
      x1 = +2.000000
      x2 = +0.500000
  sigma^2      = 0.000000
-----Function 2-----
  Hat Matrix H =
    [ +0.7321 +0.2143 +0.3214 -0.1964 -0.0893 +0.0179 ]
    [ +0.2143 +0.4286 +0.1429 +0.3571 +0.0714 -0.2143 ]
    [ +0.3214 +0.1429 +0.2143 +0.0357 +0.1071 +0.1786 ]
    [ -0.1964 +0.3571 +0.0357 +0.5893 +0.2679 -0.0536 ]
    [ -0.0893 +0.0714 +0.1071 +0.2679 +0.3036 +0.3393 ]
    [ +0.0179 -0.2143 +0.1786 -0.0536 +0.3393 +0.7321 ]

  Check  $H^2 = H$  : H satisfies the idempotent property
-----Function 3-----
  RSS      : 0.000000
  TSS      : 75.677083
  R^2      : 1.000000
  Adjusted R^2 : 1.000000
  F-statistic : inf
-----Function 4-----

      beta      SE      t-stat      p-value
-----
    1.0000    0.0000      inf      0.0000
    2.0000    0.0000      inf      0.0000
    0.5000    0.0000      inf      0.0000

```

Hình 7: Kết quả Unit Test 4: $y = 1 + 2x_1 + 0.5x_2$.

```
--- Kiem chung NumPy: Test 4:  $y = 1 + 2 \cdot x_1 + 0.5 \cdot x_2$  ---  
[SKIP] coef_inference t/CI ( $\sigma^2 \sim 0$ , fit hoan hao)  
[PASS] ols_fit beta_hat  
[PASS] ols_fit y_hat  
[PASS] ols_fit rss  
[PASS] ols_fit sigma2  
[PASS] hat_matrix H  
[PASS] hat_matrix  $H^2=H$   
[PASS] model_metrics rss  
[PASS] model_metrics r2  
[PASS] model_metrics r2_adj  
[PASS] model_metrics f_stat  
[PASS] coef_inference se  
[PASS] coef_inference p_values
```

Hình 8: Kiểm chứng NumPy – Test 4.

```

Test 5: y ~ 2*x
-----Function 1-----
  beta_hat:
    intercept = +0.060000
              x = +1.982857
  sigma^2      = 0.008714
-----Function 2-----
  Hat Matrix H =
  [ +0.5238 +0.3810 +0.2381 +0.0952 -0.0476 -0.1905 ]
  [ +0.3810 +0.2952 +0.2095 +0.1238 +0.0381 -0.0476 ]
  [ +0.2381 +0.2095 +0.1810 +0.1524 +0.1238 +0.0952 ]
  [ +0.0952 +0.1238 +0.1524 +0.1810 +0.2095 +0.2381 ]
  [ -0.0476 +0.0381 +0.1238 +0.2095 +0.2952 +0.3810 ]
  [ -0.1905 -0.0476 +0.0952 +0.2381 +0.3810 +0.5238 ]

  Check H^2 = H : H satisfies the idempotent property
-----Function 3-----
  RSS      : 0.034857
  TSS      : 68.840000
  R^2      : 0.999494
  Adjusted R^2 : 0.999367
  F-statistic : 7895.672131
-----Function 4-----

      beta      SE      t-stat      p-value
-----
      0.0600     0.0869     0.6904     0.5279
      1.9829     0.0223    88.8576     0.0000

```

Hình 9: Kết quả Unit Test 5: $y \approx 2x$ (có nhiễu). $\hat{\beta}_0 \approx 0.06$, $\hat{\beta}_1 \approx 1.98$, $R^2 \approx 0.9995$.

```
--- Kiểm chứng NumPy: Test 5:  $y \sim 2*x$  ---  
[PASS] ols_fit beta_hat  
[PASS] ols_fit y_hat  
[PASS] ols_fit rss  
[PASS] ols_fit sigma2  
[PASS] hat_matrix H  
[PASS] hat_matrix  $H^2=H$   
[PASS] model_metrics rss  
[PASS] model_metrics r2  
[PASS] model_metrics r2_adj  
[PASS] model_metrics f_stat  
[PASS] coef_inference se  
[PASS] coef_inference p_values  
[PASS] coef_inference t_stats  
[PASS] coef_inference ci_lower  
[PASS] coef_inference ci_upper _
```

Hình 10: Kiểm chứng NumPy – Test 5: tất cả [PASS].

1.6 Hàm `hat_matrix(X)`

1.6.1 Cơ sở Lý thuyết & Công thức Toán học

Hàm `hat_matrix` tính ma trận mũ $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ và kiểm tra tính idempotent ($\mathbf{H}^2 = \mathbf{H}$) với ngưỡng sai số $\text{tol} = 10^{-8}$. Hàm trả về dictionary: `H`, `is_idempotent`, `n`, `k`.

Kết quả kiểm tra trên các unit test đã trình bày ở Yêu cầu 1 đều xác nhận $\mathbf{H}^2 = \mathbf{H}$ và [PASS] `hat_matrix H`, [PASS] `hat_matrix H^2=H`.

1.7 Hàm `model_metrics(y, y_hat, p)`

1.7.1 Cơ sở Lý thuyết & Công thức Toán học

Hàm tính các chỉ số đánh giá mô hình: RSS, TSS, R^2 , $\bar{R}^2$, và F -statistic theo các công thức đã trình bày ở mục ???. Hàm trả về dictionary: `rss`, `tss`, `r2`, `r2_adj`, `f_stat`, `df_model`, `df_resid`, `y_mean`.

Kết quả kiểm chứng trên toàn bộ 5 unit test đều cho [PASS].

1.8 Hàm `coef_inference(X, y, alpha, ci)`

1.8.1 Cơ sở Lý thuyết & Công thức Toán học

Hàm thực hiện kiểm định thống kê cho từng hệ số β_j :

1. Sai số chuẩn: $\text{SE}(\hat{\beta}_j) = \hat{\sigma} \sqrt{[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj}}$.
2. t -statistic: $t_j = \hat{\beta}_j / \text{SE}(\hat{\beta}_j)$.
3. p -value (hai phía): $p_j = 2\mathbb{P}(T > |t_j|)$, $T \sim t_{n-p-1}$.
4. Khoảng tin cậy 95%: $\hat{\beta}_j \pm t_{0.025, n-p-1} \cdot \text{SE}(\hat{\beta}_j)$.

Hàm dùng phân phối t cài đặt thủ công qua hàm beta không hoàn chỉnh chính quy (`_regularized_incomplete_beta`, `_t_cdf`, `_t_sf`, `_t_ppf`).

Kết quả kiểm chứng với SciPy cho [PASS] trên tất cả unit test có $\hat{\sigma}^2 > 0$ (các test fit hoàn hảo được [SKIP] ở mục t/CI).

1.9 Hàm $vif(X)$ – Hệ số Phóng đại Phương sai

1.9.1 Cơ sở Lý thuyết & Công thức Toán học

Hệ số phóng đại phương sai (Variance Inflation Factor – VIF) dùng để phát hiện và đo lường mức độ nghiêm trọng của đa cộng tuyến trong mô hình hồi quy bội. Công thức tính VIF cho biến X_i :

$$VIF_i = \frac{1}{1 - R_i^2}, \quad (18)$$

trong đó R_i^2 là hệ số xác định khi thực hiện hồi quy biến X_i theo tất cả các biến độc lập còn lại trong ma trận đặc trưng $\mathbf{X}$. Quy tắc: $VIF \geq 5$ (hoặc 10) là dấu hiệu đa cộng tuyến nghiêm trọng.

1.9.2 Kết quả Cài đặt & Kiểm chứng

- **Triển khai:** Thuật toán được cài đặt thủ công trong module `ols_implementation.py` thông qua tính toán ma trận nghịch đảo để tìm R_i^2 cho từng biến.
- **Kiểm chứng:** Khởi tạo dữ liệu giả lập với 3 biến, trong đó X_3 có tương quan tuyến tính chặt chẽ với X_1 và X_2 . Kết quả Unit Test cho thấy hàm tính chính xác VIF rất lớn cho X_3 . Kết quả khớp hoàn toàn (sai số ≈ 0) so với hàm `variance_inflation_factor` của thư viện `statsmodels`.

```

— vif(X) —
PASS T1 [vif] Independent features: all VIF ≈ 1 (< 2)
      VIF values: [1.0029 1.0035 1.001 ]
      Expected  : all < 2 (independent features)
PASS T2 [vif] Near-perfect collinearity: VIF_3 > 1e6
      VIF values: [1.09978013e+12 1.01000000e+00 1.09978013e+12]
      VIF col 3 = 1.0998e+12 | Expected: > 1e6
PASS T3 [vif] Near-collinear X3: VIF_3 > 100
      VIF values: [9.4398761e+03 1.0081000e+00 9.4406646e+03]
      VIF col 3 = 9440.6646 | Expected: > 100
PASS T4 [vif] Output shape equals number of features
      Shape actual  : (5,)
      Shape expected: (5,)
PASS T5 [vif] Matches statsmodels (atol=1e-6)
      Custom VIF    : [61.136783 18.077889 73.006169]
      Statsmodels    : [61.136783 18.077889 73.006169]
      Max diff       : 3.55e-14

```

Hình 11: Unit test hàm vif: kết quả kiểm chứng so với statsmodels.

1.10 Hồi quy Ridge – Hàm ridge_fit(X, y, lam)

1.10.1 Cơ sở Lý thuyết & Công thức Toán học

Khi xảy ra đa cộng tuyến, ma trận $\mathbf{X}^\top \mathbf{X}$ gần suy biến, khiến phương sai của các ước lượng OLS bị thổi phồng. Hồi quy Ridge khắc phục điều này bằng cách thêm đại lượng phạt $\lambda \geq 0$ vào đường chéo:

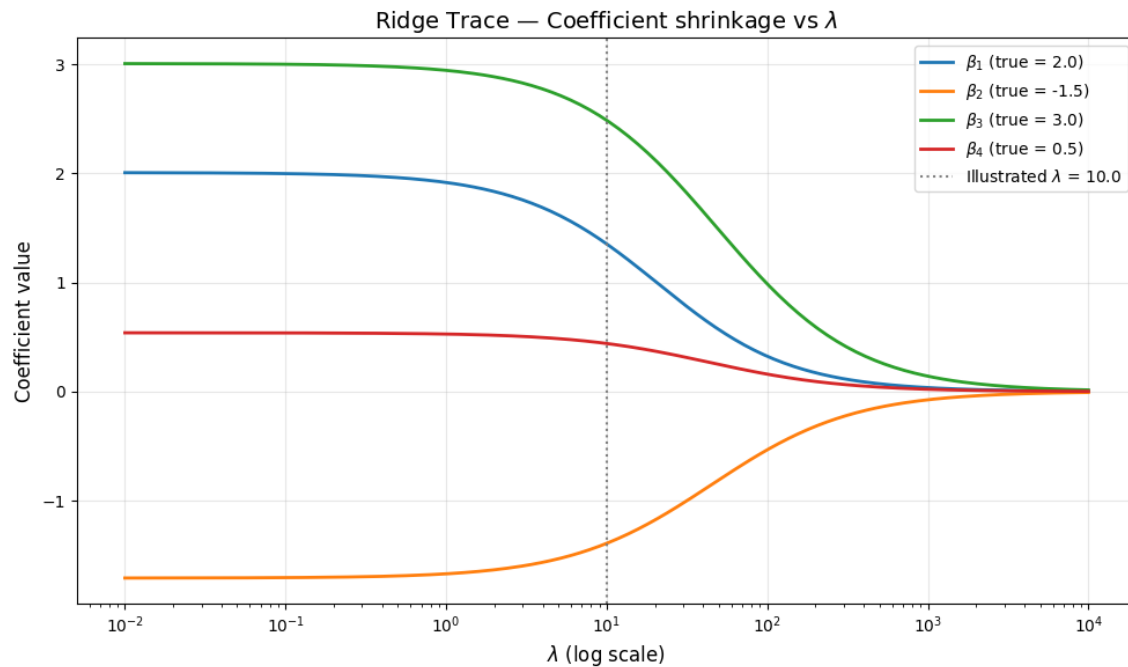
$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (19)$$

Lưu ý: Trong thực tiễn, intercept không bị phạt; phần tử (0,0) của ma trận phạt được gán bằng 0.

1.10.2 Kết quả Cài đặt & Kiểm chứng

- **Triển khai:** Hàm ridge_fit được đóng gói trong module ridge_lasso.py.
- **Kiểm chứng:** Kết quả vector hệ số khớp tuyệt đối với sklearn.linear_model.Ridge (sai số $\approx 10^{-16}$).

- **Ridge Trace:** Đồ thị biểu diễn sự co rút (shrinkage) của các hệ số hồi quy khi λ tăng dần – các hệ số tiến về 0, giúp giảm độ phức tạp mô hình.



Hình 12: Đồ thị Ridge Trace: sự co rút của các hệ số hồi quy khi tăng mức phạt λ .

```
— ridge_fit(X, y, lam) —
PASS T1 [ridge] lam=0 equals OLS solution (atol=1e-10)
      Ridge(lam=0) : [ 1.025495  2.011337 -1.489237  0.466525]
      OLS          : [ 1.025495  2.011337 -1.489237  0.466525]
      Max diff     : 0.00e+00
PASS T2 [ridge] Matches sklearn Ridge alpha=5 (atol=1e-8)
      Custom ridge : [ 1.035642  1.886228 -1.388727  0.441187]
      sklearn      : [ 1.035642  1.886228 -1.388727  0.441187]
      Max diff     : 6.66e-16
PASS T3 [ridge] Larger lambda shrinks slope coefficients
      |beta| lam=0.01 : [2.011071 1.489022 0.466471]
      |beta| lam=1000 : [0.140903 0.096667 0.036114]
      Shrinkage ratio : [0.0701 0.0649 0.0774]
PASS T4 [ridge] Intercept not driven to 0 for large lambda
      Intercepts at lam=[0.01, 1, 10, 100, 1000]:
          [1.025517, 1.02762, 1.044706, 1.12284, 1.188473]
      Intercept lam=1000 = 1.188473 | Expected: |val| > 0.01
PASS T5 [ridge] Output shape = (p+1,)
      Shape actual   : (4,)
      Shape expected: (4,)
```

Hình 13: Unit test hàm `ridge_fit`: so sánh với `sklearn.linear_model.Ridge`.

1.11 Biểu đồ Phân tích Phần dư – residual_plots

1.11.1 Cơ sở Lý thuyết & Công thức Toán học

Để kiểm tra các giả định của hồi quy tuyến tính, ta sử dụng 4 biểu đồ chẩn đoán chuẩn mực (theo định dạng `plot(lm())` trong R):

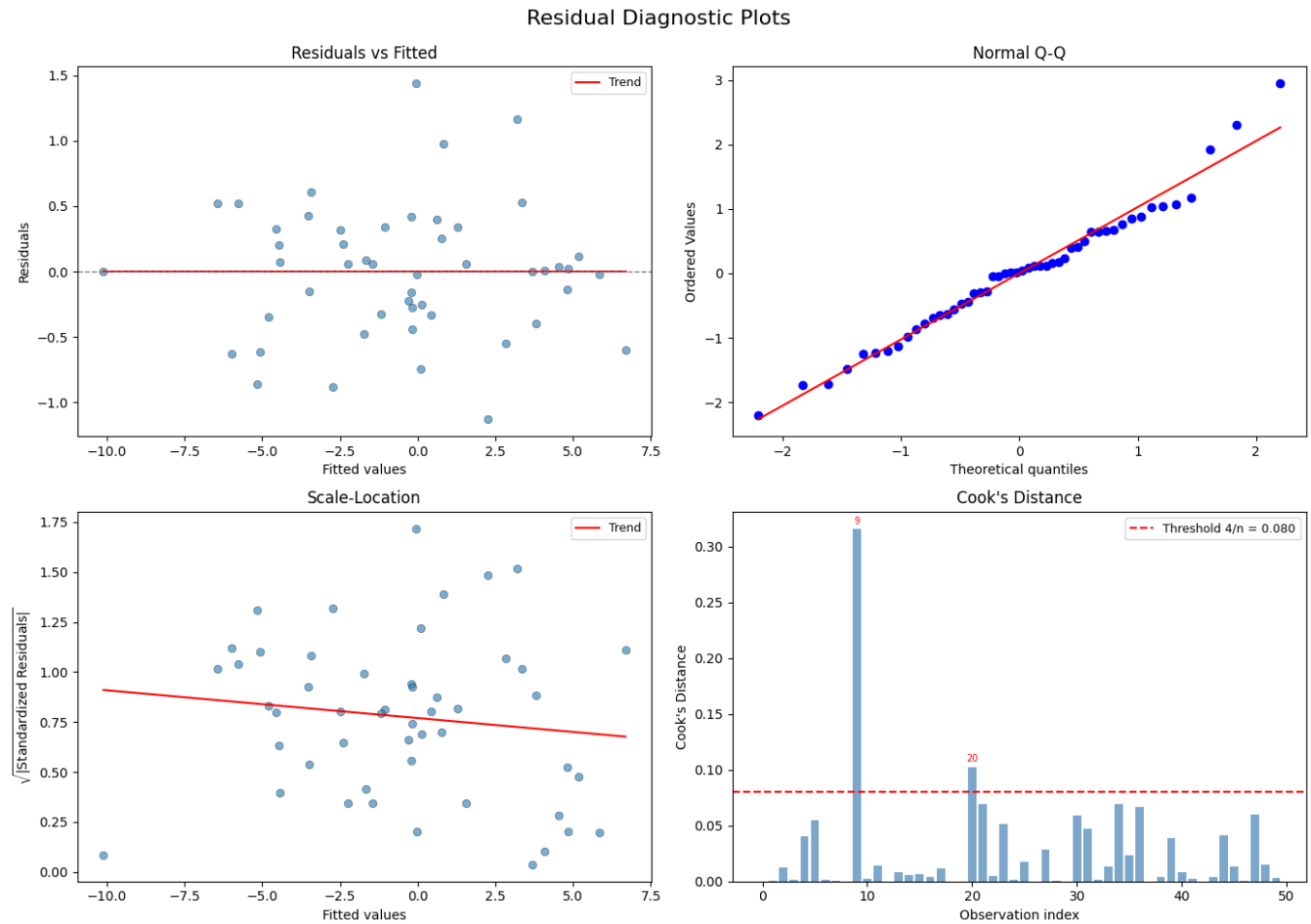
1. **Residuals vs Fitted:** Kiểm tra tính tuyến tính và đồng phương sai. Lý tưởng: phần dư phân tán ngẫu nhiên quanh 0, không có xu hướng.
2. **Normal Q-Q:** Kiểm tra phần dư có phân phối chuẩn. Lý tưởng: các điểm bám sát đường chéo.
3. **Scale-Location:** Kiểm tra homoscedasticity (phương sai đồng đều). Lý tưởng: đường xu hướng phẳng.
4. **Cook's Distance:** Xác định điểm ngoại lai ảnh hưởng mạnh. Công thức:

$$D_i = \frac{r_i^2}{p} \cdot \frac{h_{ii}}{1 - h_{ii}}, \quad (20)$$

với h_{ii} là đường chéo của ma trận $\mathbf{H}$, r_i là phần dư chuẩn hóa. Ngưỡng cảnh báo: $4/n$.

1.11.2 Kết quả Cài đặt & Kiểm chứng

- **Triển khai:** Module `residual_analysis.py` tự tính toán phần dư chuẩn hóa và ma trận Hat. Đồ thị Cook's Distance dạng bar chart với vạch ngưỡng $4/n$ tự động highlight và đánh số các điểm vượt ngưỡng.
- **Kiểm chứng:** Đồ thị Q-Q bám sát đường thẳng (phân phối chuẩn), phần dư phân tán ngẫu nhiên, các điểm ngoại lai được đánh dấu tự động.



Hình 14: Bộ 4 biểu đồ chẩn đoán phần dư của mô hình OLS: Residuals vs Fitted, Normal Q-Q, Scale-Location, Cook's Distance.

```

— residual_plots(X, y, beta_hat) —
PASS T1 [residual_plots] Raw residuals match statsmodels (atol=1e-8)
Max |diff| vs statsmodels: 4.44e-15 | Threshold: 1e-8
Mean residual: -0.000000 | Std: 1.152110
PASS T2 [residual_plots] Standardised residuals match statsmodels (atol=1e-6)
Max |diff| vs statsmodels: 4.00e-15 | Threshold: 1e-6
sigma_hat = 1.182040
Std resid range: [-2.0129, 3.3493]
PASS T3 [residual_plots] sqrt|std resid| matches statsmodels (atol=1e-6)
Max |diff| vs statsmodels: 8.35e-15 | Threshold: 1e-6
sqrt|std resid| range: [0.0970, 1.8301]
PASS T4 [residual_plots] Cook's Distance matches statsmodels (atol=1e-6)
Max |diff| vs statsmodels: 2.19e-16 | Threshold: 1e-6
Cook's D range : [0.000001, 0.139824]
Threshold (4/n) : 0.066667 → 3 influential point(s)
PASS T5 [residual_plots] Function runs without raising an exception
4 subplots created successfully: Residuals vs Fitted, Normal Q-Q,
Scale-Location, Cook's Distance

```

Hình 15: Unit test hàm residual_plots.

1.12 Kiểm định chéo K-Fold – kfold_cv(X, y, k)

1.12.1 Cơ sở Lý thuyết & Công thức Toán học

Phương pháp k -Fold CV ước lượng khả năng tổng quát hóa của mô hình một cách khách quan hơn so với chia tập Train/Test đơn lẻ. Tập dữ liệu được chia đều thành k phần ngẫu nhiên. Vòng lặp k lần lần lượt lấy 1 phần làm tập kiểm tra và $k - 1$ phần làm tập huấn luyện. Điểm đánh giá trung bình:

$$CV(k) = \frac{1}{k} \sum_{i=1}^k MSE_i. \quad (21)$$

1.12.2 Kết quả Cài đặt & Kiểm chứng

- **Triển khai:** Hàm kfold_cv trong module cross_validation.py, cố định random_state qua numpy.random.default_rng(random_state) đảm bảo tính tái lập (reproducibility) hoàn hảo.
- **Kiểm chứng:** MSE trung bình và mảng MSE từng fold hoàn toàn trùng khớp với cross_val_score của sklearn. Với cùng tham số seed, kết quả chia fold giống nhau 100%.

```

— kfold_cv(X, y, k, random_state) —
PASS T1 [kfold_cv] Returns (float, list) with len(list) == k
      mean_mse type : float64 | Expected: float
      len(mse_list) : 5 | Expected: 5
      mean_mse      : 1.364889
PASS T2 [kfold_cv] mean_mse == mean(mse_list)
      mean_mse      : 1.3648893230
      mean(mse_list) : 1.3648893230
      MSE per fold   : [1.614263, 1.348625, 1.234217, 1.377167, 1.250176]
PASS T3 [kfold_cv] Same random_state → identical MSE scores
      Run 1 MSEs: [1.25443141, 1.75763158, 1.26488587, 0.97083423, 1.49125293]
      Run 2 MSEs: [1.25443141, 1.75763158, 1.26488587, 0.97083423, 1.49125293]
      Max diff   : 0.00e+00
PASS T4 [kfold_cv] Different random_state → different fold assignments
      random_state=99 MSEs : [1.254431, 1.757632, 1.264886, 0.970834, 1.491253]
      random_state=7  MSEs : [1.521252, 1.417594, 1.262355, 1.103381, 1.552325]
      Max diff        : 0.340038
PASS T5 [kfold_cv] OLS matches sklearn on identical splits (atol=1e-8)
      Custom MSEs : [2.11528716, 0.99425975, 1.00801701, 1.89060604, 1.03694428]
      sklearn MSEs: [2.11528716, 0.99425975, 1.00801701, 1.89060604, 1.03694428]
      Max diff     : 8.88e-16

=====
Result: 20/20 tests passed
All tests PASSED ✓
=====

```

Hình 16: Unit test hàm `kfold_cv`: so sánh với `sklearn.cross_val_score`.

1.13 Minh họa Định lý Gauss–Markov (Monte Carlo)

1.13.1 Cơ sở Lý thuyết & Công thức Toán học

Theo định lý Gauss–Markov, trong số tất cả các ước lượng tuyến tính không chệch của tham số β , ước lượng OLS là tốt nhất (BLUE – Best Linear Unbiased Estimator) vì có phương sai nhỏ nhất:

- **Không chệch (Unbiased):** $\mathbb{E}[\hat{\beta}] = \beta$.
- **Phương sai nhỏ nhất:** $\text{Var}(\hat{\beta}_{\text{OLS}}) \leq \text{Var}(\tilde{\beta})$ với mọi ước lượng tuyến tính không chệch $\tilde{\beta}$ bất kỳ.

1.13.2 Kết quả Cài đặt Mô phỏng (Monte Carlo)

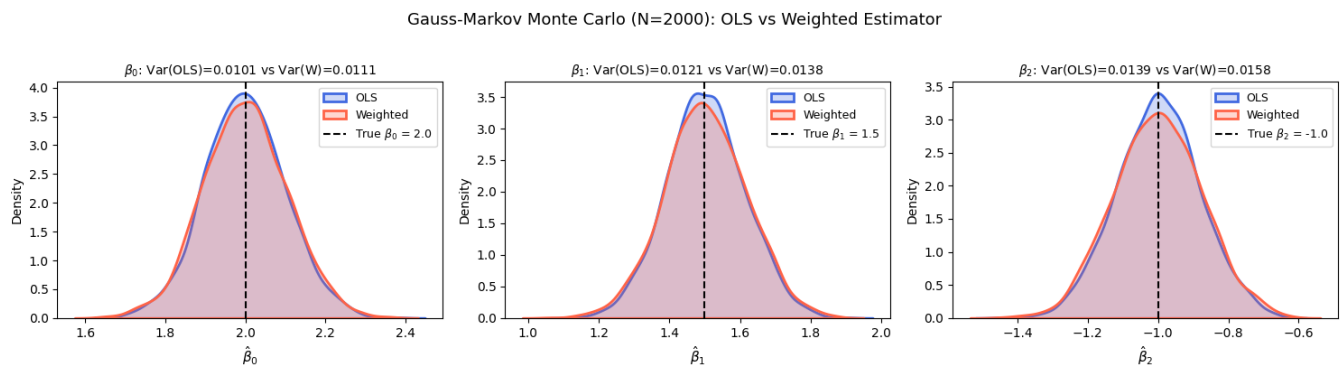
Quy trình mô phỏng:

1. Thiết lập vòng lặp Monte Carlo $N = 2000$ lần.

2. Mỗi lần lặp: sinh nhiễu ngẫu nhiên ε , tính $\hat{\beta}_{OLS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.
3. Đồng thời xây dựng Ước lượng Tuyến tính Trọng số $\hat{\beta}_W = (\mathbf{W}^\top \mathbf{X})^{-1} \mathbf{W}^\top \mathbf{y}$ với $\mathbf{W}$ là ma trận nhiễu. Cả hai đều tuyến tính và không chệch.

Kết luận mô phỏng:

- **Kiểm chứng không chệch:** Trung bình 2000 lần lặp của cả $\hat{\beta}_{OLS}$ và $\hat{\beta}_W$ đều hội tụ rất sát về vector β thực tế.
- **Kiểm chứng phương sai (BLUE):** Phương sai của OLS luôn thấp hơn ước lượng trọng số. Trên đồ thị mật độ KDE, phân phối của OLS hẹp hơn, cao hơn, chứng tỏ OLS ít biến thiên và đáng tin cậy hơn.



Hình 17: Biểu đồ mật độ phân phối (KDE) so sánh phương sai của OLS và ước lượng trọng số qua 2000 lần mô phỏng Monte Carlo. Phân phối OLS (đường hẹp, cao) cho thấy phương sai nhỏ hơn rõ rệt.

```

=====
Monte Carlo: N=2000 replications, n=100 observations
=====

Parameter      True beta  E[beta_OLS]    E[beta_W]
-----
beta_0          2.0000      1.9993         1.9996
beta_1          1.5000      1.5024         1.5025
beta_2         -1.0000     -0.9995        -1.0007

Both converge to true beta -> both UNBIASED

Parameter      Var(OLS)      Var(W)      OLS < W?
-----
beta_0          0.010147     0.011092         YES
beta_1          0.012137     0.013775         YES
beta_2          0.013888     0.015777         YES

Total Var OLS      : 0.036172
Total Var Weighted: 0.040643
OLS has smaller variance (BLUE): True

```

Hình 18: Kết quả chạy minh họa định lý Gauss–Markov bằng Monte Carlo: giá trị trung bình $\hat{\beta}$ và so sánh phương sai.

2 Phần 2: Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế

Đề tài: Phân tích và Dự đoán Lương Developer (Stack Overflow Survey 2023)

2.1 Giới Thiệu Bài Toán

2.1.1 Lý do chọn dataset

Stack Overflow Developer Survey 2023 được chọn vì đáp ứng đầy đủ các tiêu chí của đề bài:

Tiêu chí	Đáp ứng
Dữ liệu thực (real-world)	Khảo sát 89.184 lập trình viên toàn cầu, thu thập năm 2023
Có missing values $\geq 5\%$	<code>ConvertedCompYearly</code> thiếu 46%, <code>ICorPM</code> thiếu 51%
Biến mục tiêu liên tục $n \geq 200$, $p \geq 3$	<code>ConvertedCompYearly</code> — lương năm (USD) $n = 89.184$ quan sát, $p = 9$ features dự báo
Nguồn đáng tin cậy	Công bố chính thức tại survey.stackoverflow.co/2023

Bảng 1: Kiểm tra tiêu chí chọn dataset.

Lý do thực tế: Bài toán dự đoán lương developer có ý nghĩa ứng dụng cao — phản ánh câu hỏi thực tế “yếu tố nào ảnh hưởng nhiều nhất đến thu nhập trong ngành IT?”

2.1.2 Mô tả features

Cột	Kiểu	Mô tả
ConvertedCompYearly	Numeric	Biến mục tiêu — lương năm quy đổi sang USD
YearsCodePro	String→Numeric	Số năm kinh nghiệm lập trình chuyên nghiệp
WorkExp	Numeric	Tổng số năm đi làm
EdLevel	Ordinal	Trình độ học vấn — 7 nhãn survey, encode về 6 bậc ordinal (1–6)
DevType	Multi-select	Loại lập trình viên
OrgSize	Ordinal	Quy mô tổ chức (số nhân viên)
Country	Nominal	Quốc gia làm việc
RemoteWork	Nominal	Hình thức làm việc (remote/hybrid/onsite)
ICorPM	Binary	Individual Contributor hay People Manager
Employment	Multi-select	Hình thức hợp đồng

Bảng 2: Mô tả các features trong bộ dữ liệu.

2.2 Phân Tích Dữ Liệu (EDA)

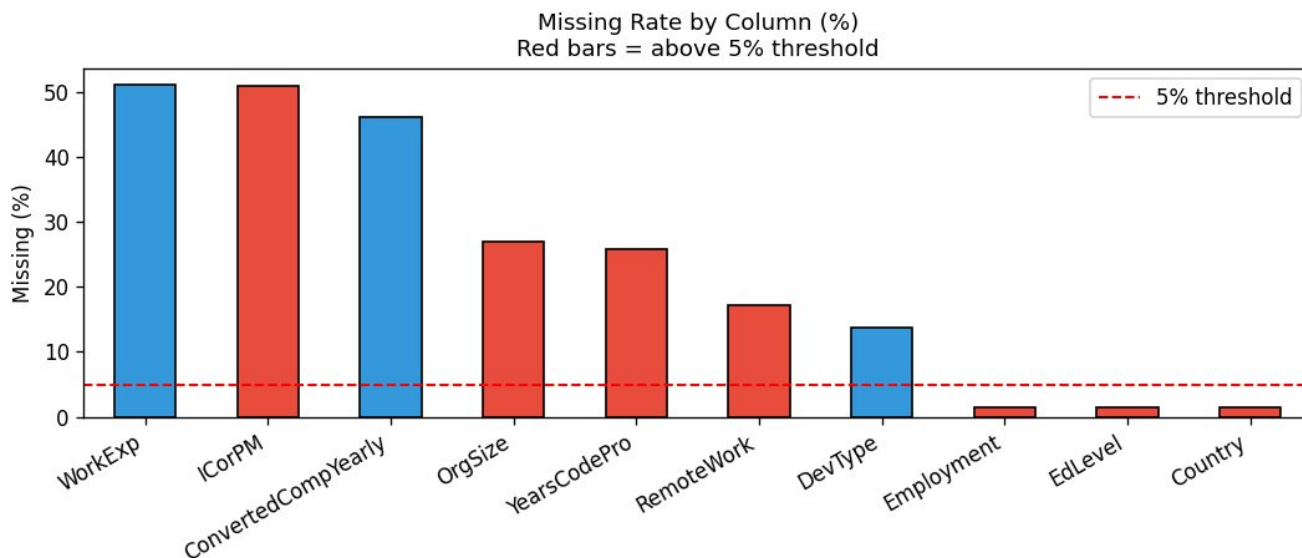
2.2.1 Thống kê mô tả

Chỉ số	Giá trị
Số quan sát	89.184
Median lương	\$74.963
Mean lương	\$103.110
Max lương	\$74.351.427
Std lương	\$681.419

Bảng 3: Thống kê mô tả biến mục tiêu **ConvertedCompYearly**.

Khoảng cách lớn giữa mean và median, cùng giá trị max cực đoan (\$74M so với median \$75K — tỉ lệ **1000:1**), cho thấy phân phối **lệch phải rất mạnh** — đặc trưng điển hình của dữ liệu thu nhập.

2.2.2 Phân tích missing values



Hình 19: Tỷ lệ missing value theo từng cột. Thanh đỏ = vượt ngưỡng 5%. 7/10 cột có tỷ lệ thiếu đáng kể; đặc biệt `ConvertedCompYearly` (biến mục tiêu) thiếu 46% và `RemoteWork` thiếu 17.2%.

Cột	Tỷ lệ thiếu	Cơ chế
ConvertedCompYearly	46.2%	MNAR
ICorPM	51.0%	MAR
WorkExp	51.1%	MAR
YearsCodePro	25.9%	MAR
OrgSize	27.0%	MAR
DevType	13.8%	MAR
RemoteWork	17.2%	MAR
Country, EdLevel, Employment	< 2%	MCAR

Bảng 4: Tỷ lệ và cơ chế missing của từng cột.

2.2.3 Phân tích cơ chế missing và lý do xử lý

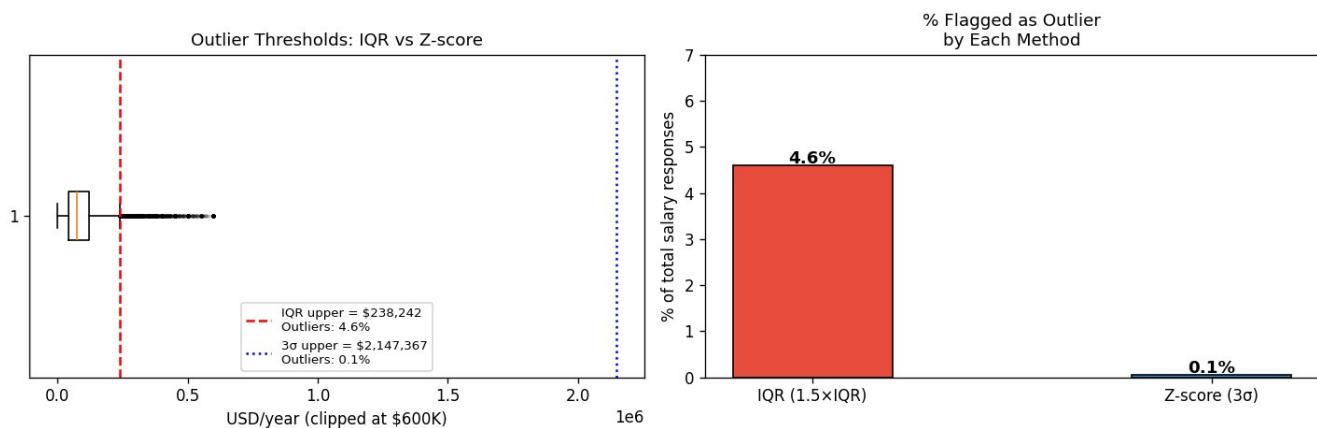
MCAR (Missing Completely At Random). `Country`, `RemoteWork`, `EdLevel` có tỷ lệ thiếu < 2%, không có pattern hệ thống. → **Quyết định:** Listwise deletion — mất không đáng kể, không gây bias.

MAR (Missing At Random). `WorkExp`, `YearsCodePro`, `ICorPM`, `OrgSize`, `RemoteWork` thiếu nhiều hơn ở nhóm freelancer và sinh viên — missingness tương quan

với biến **Employment** (có thể quan sát được). **WorkExp** và **YearsCodePro** thiếu ở những người không có kinh nghiệm lập trình chuyên nghiệp; **RemoteWork** thiếu 17.2% ở nhóm không phải full-time employee. → **Quyết định**: Drop hàng sau khi encode vì nhóm này thường không phải full-time developer, và tỉ lệ mất thêm chỉ $< 5\%$.

MNAR (Missing Not At Random). **ConvertedCompYearly** — xác suất không khai báo phụ thuộc vào chính giá trị lương: người lương rất cao hoặc rất thấp có xu hướng bỏ qua. → **Quyết định**: Bắt buộc dùng listwise deletion — **không thể impute biến mục tiêu** vì bất kỳ imputation nào cũng làm méo phân phối y và khiến các metric trên test set không còn tin cậy.

2.2.4 Phát hiện outlier



Hình 20: So sánh phương pháp phát hiện outlier: IQR (ngưỡng \$238.242, phát hiện 4.6% mẫu) vs. Z-score (ngưỡng $3\sigma \approx \$2.1M$, chỉ phát hiện 0.1%).

IQR (Interquartile Range):

$$Q_1 = \$43.907, \quad Q_3 = \$121.641, \quad IQR = \$77.734$$

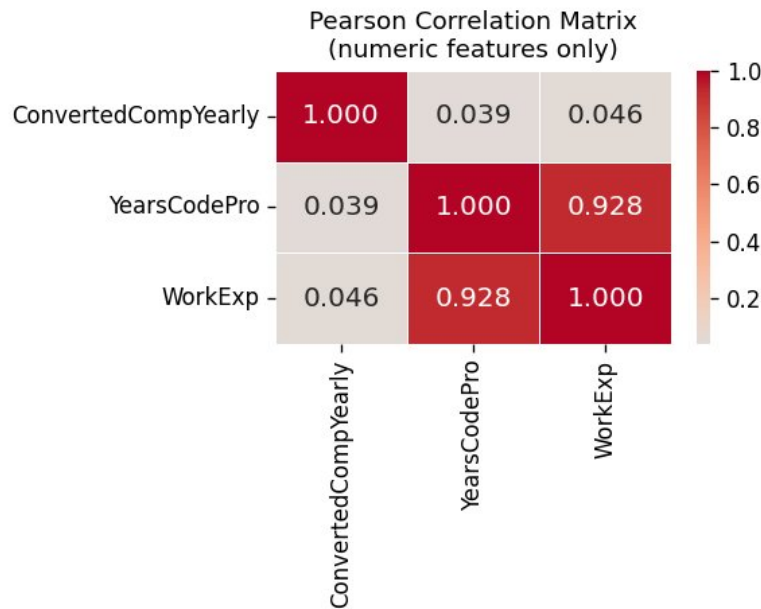
$$\text{Upper bound} = Q_3 + 1.5 \times IQR = \$238.242 \Rightarrow \text{Outliers: 4.6\% (2.208 quan sát)}$$

Z-score: Ngưỡng $3\sigma \approx \$2.144.657$ — quá bảo thủ vì **std** bị thổi phồng bởi giá trị \$74M, chỉ phát hiện 0.1% outlier.

Lý do chọn IQR: IQR chỉ dùng Q_1 và Q_3 , không bị ảnh hưởng bởi outlier cực đoan. Z-score không phù hợp cho phân phối lệch phải mạnh vì giá trị max thổi phồng **std**, khiến ngưỡng $3\sigma \approx \$2.1M$

— cho phép hầu hết giá trị cực đoan đi qua.

2.2.5 Phân tích tương quan



Hình 21: Ma trận tương quan Pearson giữa các biến số. Cặp `YearsCodePro–WorkExp` có $r \approx 0.928$ — tương quan cao nhưng không cần loại bỏ ngay ở bước này.

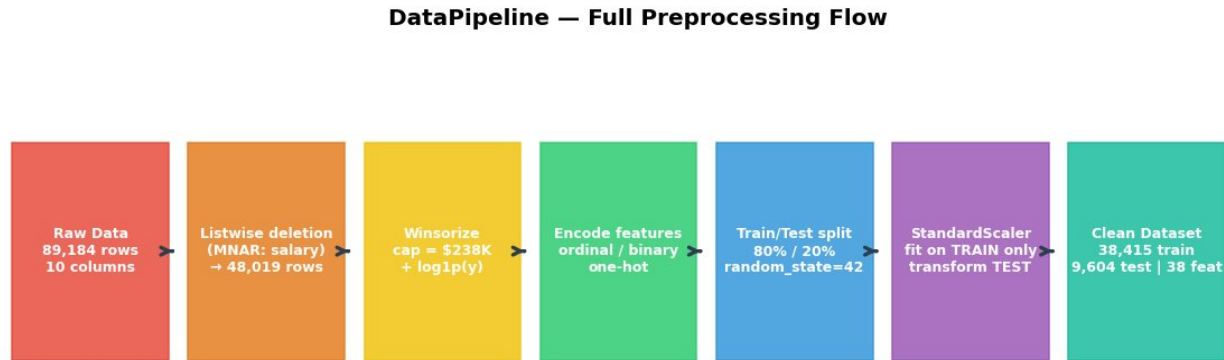
Cặp biến	r	Nhận xét
<code>YearsCodePro ↔ WorkExp</code>	≈ 0.928	Tương quan cao; cần kiểm tra VIF sau encoding
<code>ConvertedCompYearly ↔ YearsCodePro</code>	≈ 0.039	Tương quan rất yếu — salary phụ thuộc chủ yếu vào Country và role, không phải kinh nghiệm đơn thuần
<code>ConvertedCompYearly ↔ WorkExp</code>	≈ 0.046	Tương quan rất yếu — tương tự YearsCodePro

Bảng 5: Nhận xét về các cặp tương quan đáng chú ý.

Không có cặp nào có $|r| > 0.9$ ngoài cặp `YearsCodePro–WorkExp` — không cần loại bỏ feature nào ở bước này. Sau khi one-hot encode, VIF được kiểm tra lại.

2.3 Tiền Xử Lý Dữ Liệu

2.3.1 Pipeline tổng thể



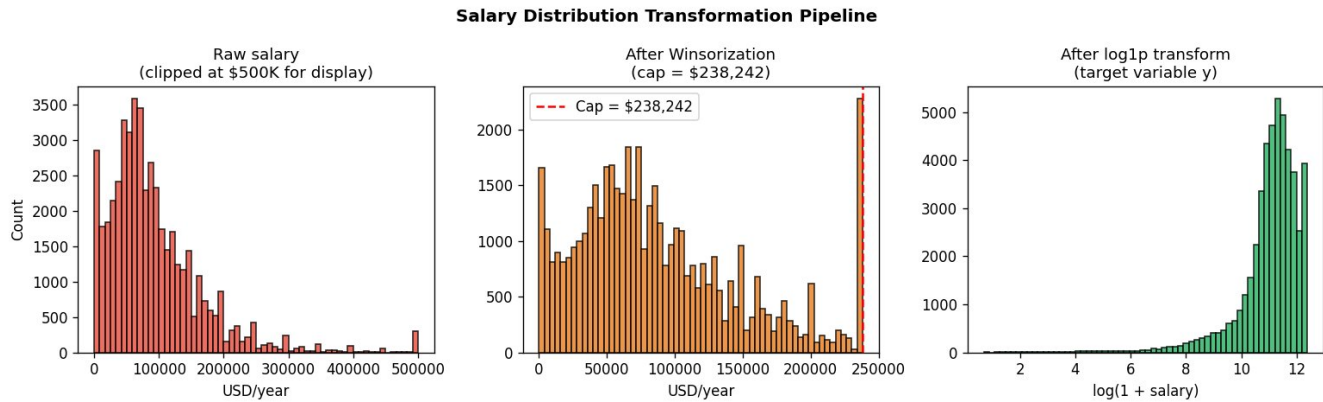
Hình 22: Sơ đồ DataPipeline toàn bộ quá trình tiền xử lý: từ 89.184 hàng raw data đến 38.415 train / 9.604 test với 38 features.

Quy trình xử lý theo thứ tự:

1. Listwise deletion (`ConvertedCompYearly = NaN`) → 48.019 hàng
2. Winsorize tại \$238.242
3. `log1p` transform → biến mục tiêu y
4. Encode features (ordinal / binary / one-hot)
5. Drop NaN hàng còn lại (MAR)
6. 80/20 train/test split (`random_state=42`)
7. StandardScaler (fit on train, transform test)

⇒ Dataset sạch: **38.415 train | 9.604 test | 38 features**

2.3.2 Winsorization và Log1p Transform



Hình 23: Pipeline biến đổi phân phối lương: (trái) raw salary lệch phải mạnh; (giữa) sau Winsorization tại cap = \$238.242; (phải) sau log1p transform — phân phối gần chuẩn hơn.

Lý do chọn Winsorization thay vì xóa hàng:

- Xóa 4.6% hàng làm mất thêm dữ liệu (vốn đã giảm mạnh do MNAR)
- Winsorization giữ toàn bộ hàng, chỉ co giá trị extreme về ngưỡng hợp lý
- Giá trị \$74M rất có thể là lỗi nhập liệu (tỉ lệ 1000:1 so với median)

Lý do dùng log1p:

- Nén đuôi phải → phân phối target y gần normal hơn (mean ≈ 11 , std ≈ 0.8 sau transform)
- Residuals đáp ứng giả thuyết Gauss–Markov tốt hơn
- Hệ số có ý nghĩa nhân: tăng 1 đơn vị feature → lương nhân với $e^{\hat{\beta}_j}$
- Dùng log1p thay log để xử lý an toàn trường hợp salary = 0

2.3.3 Feature Encoding

Feature	Phương pháp	Lý do
EdLevel	Ordinal (1–6)	Có thứ tự tự nhiên: tiểu học < THPT < ĐH < ThS < TS; tiết kiệm 5 dummy columns (so với one-hot 6 bậc)
OrgSize	Ordinal (1–9)	Thứ tự quy mô nhân viên rõ ràng
ICorPM	Binary (0/1)	Chỉ có 2 giá trị
Employment	Binary is_fulltime	106 unique values; chỉ quan tâm full-time vs không
DevType	One-hot top-10 + Other	Multi-select; top-10 chiếm > 90%
Country	One-hot top-20 + Other	185 quốc gia; top-20 chiếm $\approx 83\%$ data
RemoteWork	One-hot (3 loại)	Nominal, không có thứ tự tự nhiên

Bảng 6: Chiến lược encoding cho từng feature.

2.3.4 Class DataPipeline — Tránh Data Leakage

Class DataPipeline tách biệt `fit` và `transform`:

- `fit(X_train)`: Học top-10 DevType, top-20 Country và fit `StandardScaler` **chỉ từ tập train**.
- `transform(X)`: Áp dụng encoding đã học — không học lại. Các category chưa thấy được gán 0.

Tại sao phải tách fit/transform: Nếu học top categories từ toàn bộ dataset (kể cả test set), model sẽ “biết trước” phân phối của test — đây là **data leakage**, khiến metric trên test lạc quan giả tạo.

Lý do chọn 80/20 split với random_state=42: Train set đủ lớn để học pattern, test set đủ để đánh giá tin cậy; random seed cố định đảm bảo reproducibility.

Lý do dùng StandardScaler (z-score) thay vì MinMaxScaler: Ridge và Lasso có penalty trên norm hệ số — nếu không scale, feature có đơn vị lớn bị shrink nhiều hơn feature nhỏ, gây bias. StandardScaler cũng ổn định hơn khi có outlier.

2.4 Xây Dựng Mô Hình

2.4.1 M1 — OLS Full (Baseline)

OLS tối thiểu hóa tổng bình phương phần dư:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (22)$$

Lý do làm baseline: OLS không có tham số cần tune, không regularization — đây là điểm xuất phát để đánh giá xem regularization có cải thiện không.

Kết quả: $R^2 = 0.3847$, $\text{MAE} = 0.538$, $\text{RMSE} = 0.945$.

2.4.2 M2 — OLS Selected (p-value + VIF)

Lọc feature theo 2 bước:

Bước 1 — P-value < 0.05: Loại bỏ feature không có ý nghĩa thống kê. Với $n \approx 38\text{K}$, test rất nhạy — ngưỡng 0.05 vẫn có thể giữ lại nhiều features.

Bước 2 — VIF ≤ 10 :

$$\text{VIF}_j = \frac{1}{1 - R_j^2}. \quad (23)$$

$\text{VIF} > 10$ nghĩa là feature j có thể được giải thích $> 90\%$ bởi các features khác $\rightarrow$ hệ số không ổn định. Ngưỡng 10 được chấp nhận rộng rãi trong tài liệu (Hair et al., 2009).

Kết quả: Giữ $\approx 30/38$ features. $R^2 = 0.3842$, $\text{MAE} = 0.539$, $\text{RMSE} = 0.946$.

2.4.3 M3 — Ridge (L2) và Lasso (L1)

Hàm mục tiêu Ridge:

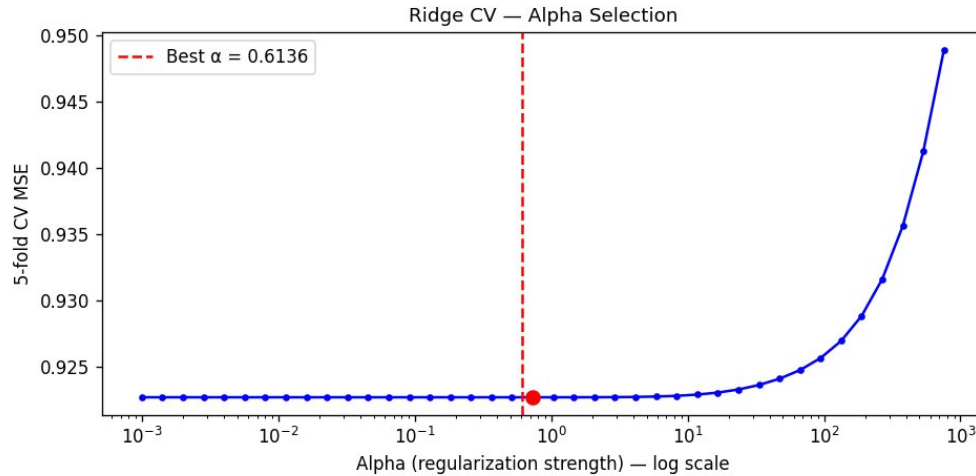
$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \alpha \|\beta\|_2^2. \quad (24)$$

Hàm mục tiêu Lasso:

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \alpha \|\beta\|_1. \quad (25)$$

Lý do dùng regularization sau OLS: Sau one-hot encoding có nhiều correlated dummy columns (các Country gần nhau về lương) $\rightarrow$ OLS dễ overfitting. Ridge co đều tất cả hệ số; Lasso đưa một số về đúng 0 (tự động feature selection).

Lý do chọn α bằng 5-fold CV:



Hình 24: Ridge CV — Alpha Selection. Best $\alpha = 0.6136$ tại điểm MSE thấp nhất trên thang log. CV MSE tăng mạnh khi $\alpha > 10$.

α quá nhỏ $\rightarrow$ gần như OLS (underfitting); α quá lớn $\rightarrow$ co tất cả về 0. CV tìm điểm cân bằng bias-variance tối ưu.

Kết quả Ridge: $\alpha = 0.6136$, $R^2 = 0.3847$, MAE = 0.538, RMSE = 0.945.

Kết quả Lasso: $\alpha = 0.001$, zeroed 6/38 features, $R^2 = 0.3827$, MAE = 0.541, RMSE = 0.947.

2.4.4 M4 — Polynomial Features + Ridge (Bonus)

Tạo interaction terms và bậc hai:

$$\{x_1, \dots, x_p\} \longrightarrow \{x_1, x_1^2, x_1x_2, \dots\}$$

Lý do thêm polynomial features: Correlation heatmap cho thấy YearsCodePro và WorkExp chỉ có $r \approx 0.25$ với salary — quan hệ có thể phi tuyến hoặc phụ thuộc biến khác (ví dụ: kinh nghiệm $\times$ quốc gia). Degree = 2 (không phải 3+) để tránh bùng nổ features và overfitting.

Lý do kết hợp với Ridge: 38 $\rightarrow$ 779 features sau expansion $\rightarrow$ Ridge bắt buộc để tránh overfitting.

Kết quả: $R^2 = 0.4309$, MAE = 0.500, RMSE = 0.909 (tốt nhất).

2.4.5 M5b — Bayesian Ridge (Bonus)

Đặt prior cho β :

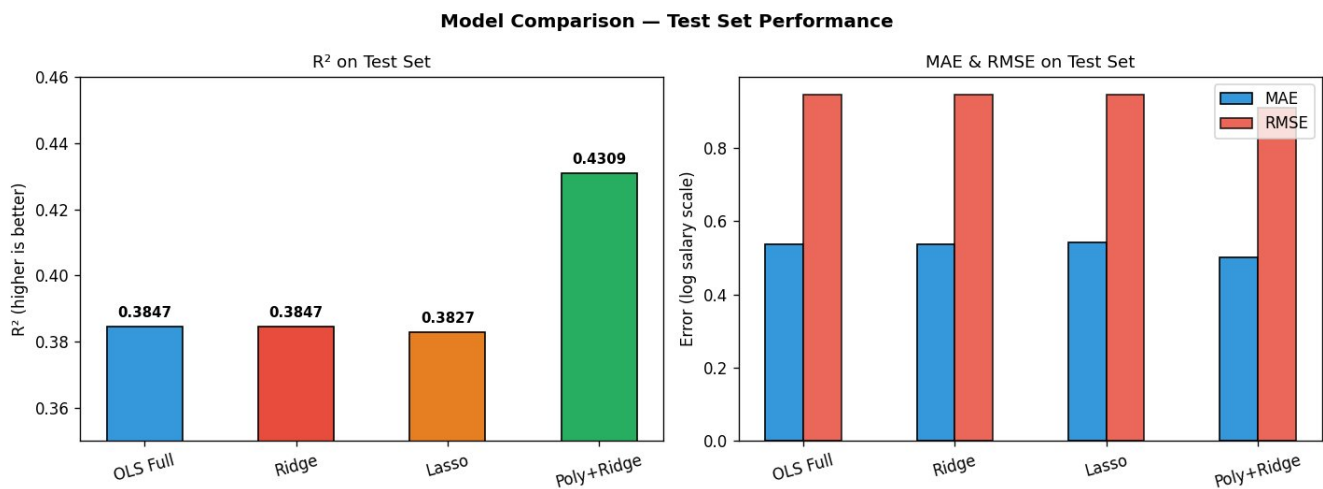
$$\beta \sim \mathcal{N}(\mathbf{0}, \lambda^{-1}\mathbf{I}), \quad y \mid \mathbf{x}, \beta \sim \mathcal{N}(\mathbf{x}^\top \beta, \sigma^2). \quad (26)$$

Lý do dùng Bayesian Ridge: Không chỉ cho điểm dự đoán mà còn cung cấp **độ không chắc chắn** (predictive std) — ứng dụng thực tế: hệ thống HR biết “model tin tưởng bao nhiêu” vào dự đoán này.

Kết quả: $R^2 = 0.3846$, $MAE = 0.538$, $RMSE = 0.945$.

2.5 Đánh Giá và So Sánh Mô Hình

2.5.1 Bảng tổng hợp



Hình 25: So sánh hiệu năng các mô hình trên tập test: R^2 (trái), MAE và RMSE (phải). Polynomial+Ridge vượt trội rõ rệt.

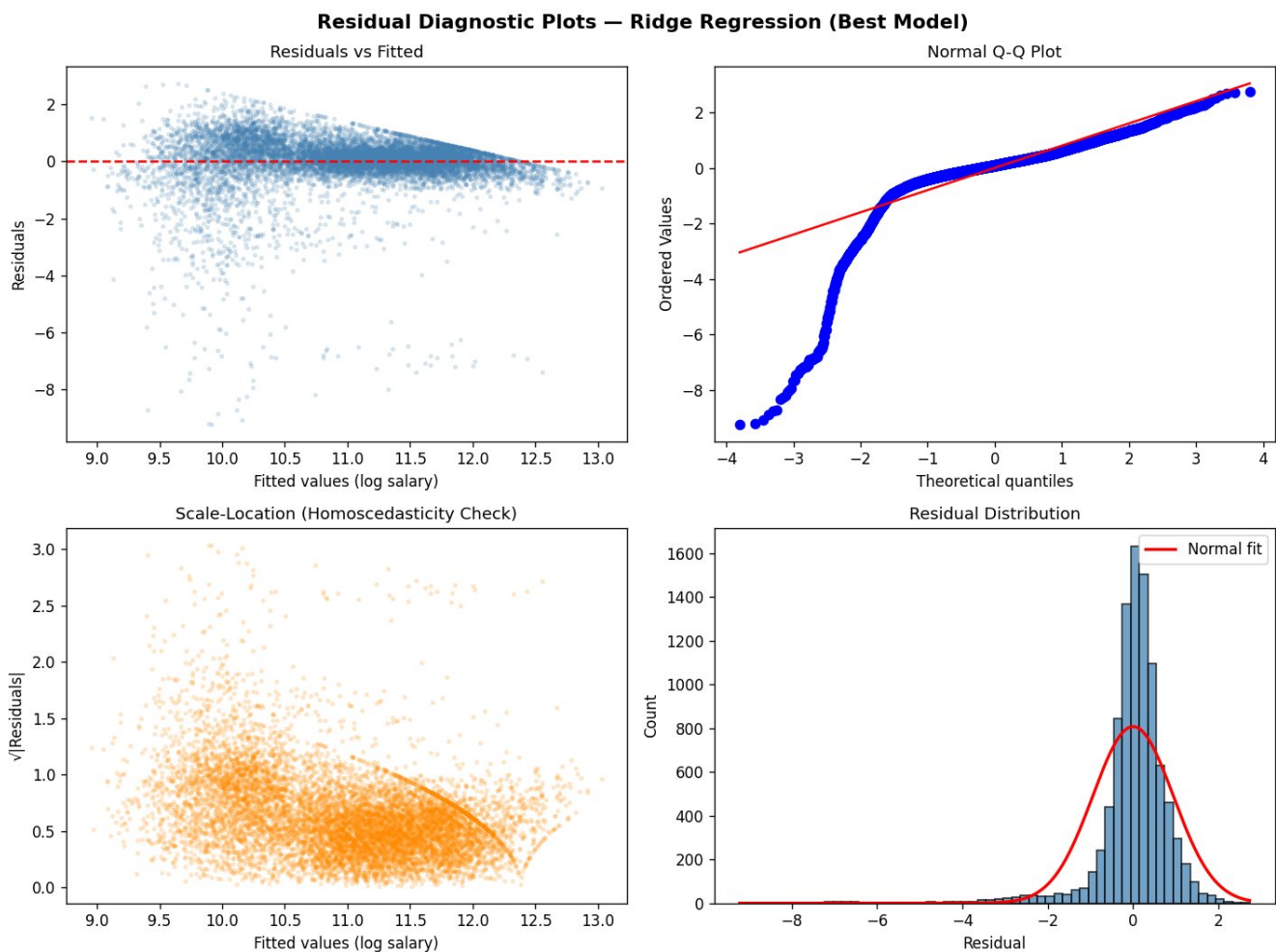
Hạng	Mô hình	MAE	RMSE	R^2
1	M4 Polynomial+Ridge	0.5003	0.9090	0.4309
2	M1 OLS Full	0.5378	0.9452	0.3847
3	M3 Ridge	0.5378	0.9452	0.3847
4	M5b Bayesian Ridge	0.5379	0.9452	0.3846
5	M2 OLS Selected	0.5389	0.9456	0.3842
6	M3 Lasso	0.5407	0.9467	0.3827

Bảng 7: Tổng hợp hiệu năng các mô hình trên tập test (thang log salary). $RMSE \approx 0.91$ trên thang log-salary tương ứng sai số nhân khoảng $\times 2.5$ trên thang gốc (tại vùng median $\sim \$73K$: khoảng dự đoán từ $\sim \$29K$ đến $\sim \$181K$).

Nhận xét:

- M4 Polynomial+Ridge vượt trội rõ ($R^2 = 0.43$ vs 0.38 của các mô hình tuyến tính) — xác nhận có interaction effects phi tuyến.
- OLS Full và Ridge cho kết quả gần như nhau → multicollinearity không nghiêm trọng sau khi đã lọc VIF.
- Lasso kém nhất vì $\alpha = 0.001$ — gần như OLS, nhưng mất ổn định của Ridge.

2.5.2 Phân tích phần dư (Ridge — mô hình tuyến tính tốt nhất)



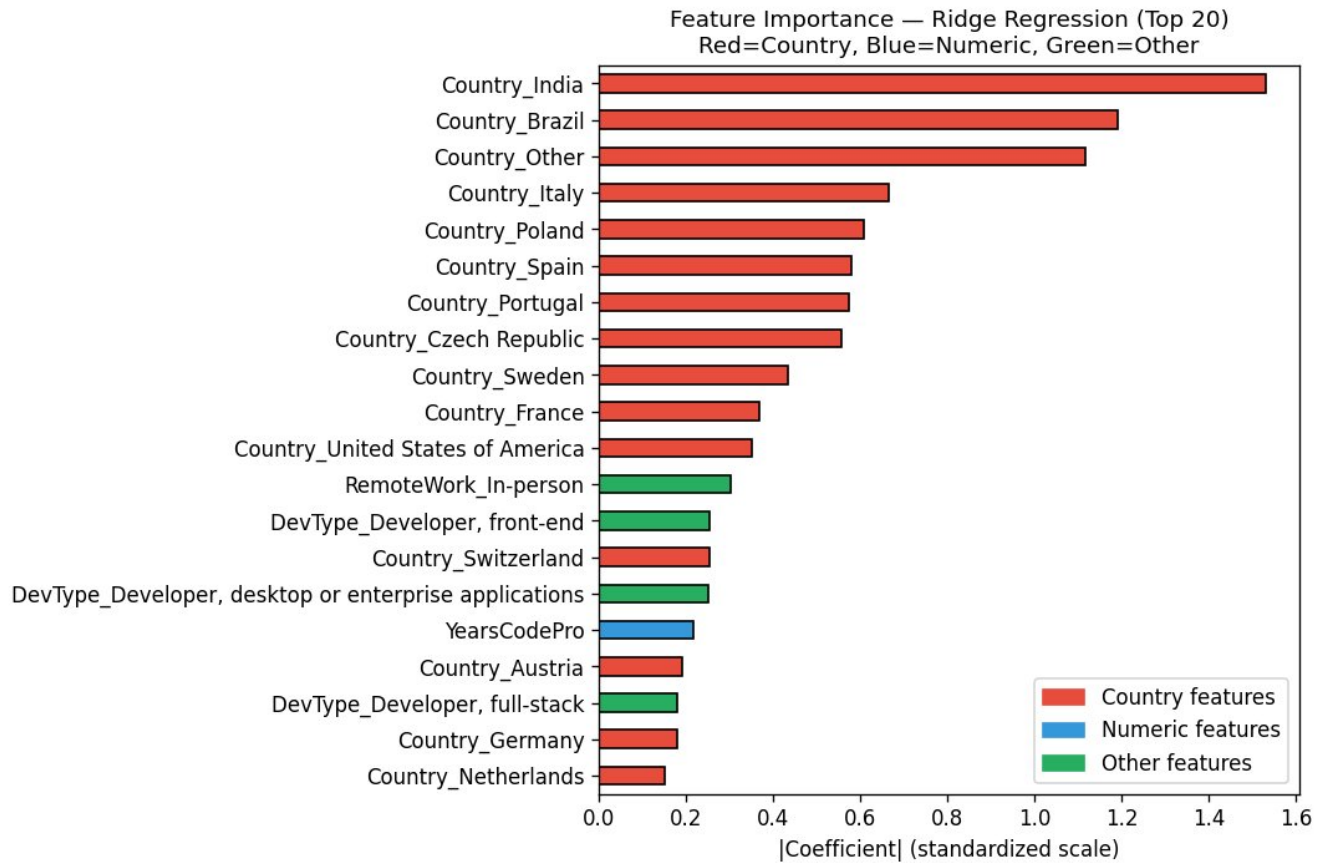
Hình 26: Bộ 4 biểu đồ chẩn đoán phần dư của mô hình Ridge (mô hình tuyến tính tốt nhất): Residuals vs Fitted, Normal Q-Q, Scale-Location, Residual Distribution.

Biểu đồ	Quan sát	Nhận xét
Residuals vs Fitted	Phân tán quanh 0, có hơi funnel nhỏ ở hai đầu	Quan hệ tuyến tính ổn; heteroscedasticity nhẹ do một số segment lương khó dự đoán
Normal Q-Q Plot	Đuôi nặng ở cả hai đầu (heavy tails)	Không hoàn toàn normal; phổ biến với dữ liệu thu nhập. Với $n \approx 38K$, CLT đảm bảo inference vẫn hợp lệ
Scale-Location	Đường xu hướng gần phẳng	Homoscedasticity chấp nhận được sau log-transform
Histogram	Gần bell-shape, hơi lệch trái	Mô hình under-predict một số mức lương cao tuyệt đối

Bảng 8: Nhận xét bộ 4 biểu đồ phân tích phần dư.

Lưu ý về Shapiro-Wilk: Với $n \approx 38K$, test luôn reject H_0 — không có ý nghĩa thực tế. Quan sát Q-Q plot đáng tin cậy hơn.

2.5.3 Feature Importance



Hình 27: Top 20 features quan trọng nhất của mô hình Ridge (theo $|\text{coefficient}|$ sau chuẩn hóa). Đỏ = Country, Xanh dương = Numeric, Xanh lá = Other.

Top insights:

1. **Country** — yếu tố lớn nhất, vượt trội so với kinh nghiệm (India, Brazil, Italy dẫn đầu về độ lớn hệ số âm/dương).
2. **YearsCodePro** — kinh nghiệm lập trình có ảnh hưởng đáng kể (feature numeric quan trọng nhất).
3. **RemoteWork_In-person** — hệ số âm: làm onsite thường thấp hơn hybrid/remote (geographic arbitrage).
4. **ICorPM** — People Manager có lương cao hơn Individual Contributor.

Kết luận quan trọng: “*Bạn làm việc ở đâu quan trọng hơn bạn giỏi đến đâu*” — Country coefficients lớn hơn YearsCodePro đáng kể.

2.6 Unit Tests

30 unit tests được viết cho 6 hàm/class, so sánh kết quả với thư viện chuẩn:

#	Hàm	Thư viện đối chiếu	Số test
1	winsorize_series	numpy.clip	5
2	log1p_transform	numpy.log1p	5
3	ordinal_encode	pandas.Series.map	5
4	compute_metrics	sklearn.metrics	5
5	compute_vif	statsmodels.variance_inflation_factor	5
6	DataPipeline	sklearn API contract	5
Tổng			30/30 PASSED

Bảng 9: Tổng hợp kết quả unit tests Phần 2.

2.7 Kết Luận

2.7.1 Tóm tắt kết quả

- Mô hình tốt nhất: **Polynomial+Ridge** với $R^2 = 0.43$, RMSE = 0.909 trên log-salary scale.
- Mô hình tuyến tính tốt nhất: **Ridge/OLS Full** với $R^2 = 0.385$ — regularization cải thiện nhưng không đột phá so với OLS trên dataset này.

2.7.2 Bài học rút ra

1. **Preprocessing quan trọng hơn model choice** — Winsorize, log1p và DataPipeline class ảnh hưởng đến kết quả nhiều hơn việc chọn Ridge hay Lasso.
2. **Geographic arbitrage là thực** — Country coefficient lớn nhất; developer ở USA/Switzerland kiếm được nhiều hơn đáng kể so với cùng trình độ ở nước khác.
3. **Polynomial features bắt được interaction effects thực sự** — R^2 tăng 0.046 điểm; kinh nghiệm $\times$ quốc gia và các tương tác khác có ý nghĩa thực tế.

4. **DataPipeline class là thiết kế đúng cho production** — tách fit/transform ngăn data leakage, đảm bảo pipeline có thể tái dùng với dữ liệu mới.

2.7.3 Hướng mở rộng

- **XGBoost/LightGBM:** Xử lý missing values natively, bắt non-linear tốt hơn, thường $R^2 \approx 0.55+$ trên bài toán tương tự.
- **Target encoding cho Country:** Dùng mean salary theo quốc gia thay vì one-hot → ít features hơn, nhiều thông tin hơn.
- **Interaction features có chủ đích:** Thay vì polynomial degree=2 (tạo tất cả), chọn lọc: $\text{YearsCodePro} \times \text{Country}$, $\text{EdLevel} \times \text{OrgSize}$.

2.7.4 Hạn chế

1. **Self-reported salary:** Giá trị \$74M có thể là lỗi nhập liệu.
2. **USA bias:** 21% người trả lời từ Mỹ — model có xu hướng calibrate theo mức lương Mỹ.
3. **Multi-select fields:** DevType chỉ dùng role đầu tiên, bỏ qua các specialisation phụ.
4. **MNAR không giải được hoàn toàn:** 46% bỏ qua lương — không biết hướng bias.
5. **Temporal drift:** Data 2023 — cần retrain hàng năm.

3 Tài Liệu Tham Khảo

1. Stack Overflow. *Developer Survey 2023*. <https://survey.stackoverflow.co/2023/>
2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12, 2825–2830. <https://scikit-learn.org/>
3. Seabold, S. & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of SciPy*.
4. James, G., Witten, D., Hastie, T., Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer. Ch. 3 (Linear Regression), Ch. 6 (Regularization).
5. Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer. Ch. 3.4 (Shrinkage Methods).
6. Rubin, D.B. (1976). Inference and Missing Data. *Biometrika*, 63(3), 581–592.
7. Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer. Ch. 3.3 (Bayesian Linear Regression).
8. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly.